

The topology of software risk in scientific models

4. Scalability benchmark

Arnald Puy

Contents

1	Preliminary	2
2	Synthetic graph generator	2
3	Configurations	3
4	Benchmark function	4
5	Run benchmark	7
6	Plots	9
7	Session information	12

1 Preliminary

```
# SCALABILITY BENCHMARK USING softwareRisk FUNCTIONS #####
#####

sensobol::load_packages(c("data.table", "tidyverse", "openxlsx", "scales",
                          "cowplot", "tidygraph", "igraph", "parallel",
                          "sensobol", "softwareRisk", "benchmarkme"))

# Set seed -----

set.seed(123)

# Import theme -----

theme_AP <- softwareRisk::theme_AP
```

2 Synthetic graph generator

```
# SYNTHETIC CALL-GRAPH GENERATOR #####

make_scalability_dag <- function(n_entry, n_mid_layer, n_layers, n_sink,
                                p_extra = 0.02, p_skip = 0.01,
                                cc_shape = 2, cc_rate = 0.1, seed = 1) {

  set.seed(seed)

  layers <- vector("list", n_layers + 2L)
  id <- 0L

  layers[[1L]] <- (id + 1L):(id + n_entry)
  id <- id + n_entry

  for (l in seq_len(n_layers)) {
    layers[[l + 1L]] <- (id + 1L):(id + n_mid_layer)
    id <- id + n_mid_layer
  }

  layers[[n_layers + 2L]] <- (id + 1L):(id + n_sink)
  id <- id + n_sink

  edges_list <- list()

  for (l in seq_len(n_layers + 1L)) {
    from <- layers[[l]]
    to   <- layers[[l + 1L]]
```

```

spine <- cbind(from, sample(to, length(from), replace = TRUE))
edges_list[[length(edges_list) + 1L]] <- spine

if (p_extra > 0) {
  pairs <- expand.grid(from = from, to = to)
  keep <- runif(nrow(pairs)) < p_extra
  if (any(keep)) {
    edges_list[[length(edges_list) + 1L]] <- as.matrix(pairs[keep, ])
  }
}

if (p_skip > 0 && 1 <= n_layers) {
  to2 <- layers[[1 + 2L]]
  add_skip <- runif(length(from)) < p_skip
  if (any(add_skip)) {
    edges_list[[length(edges_list) + 1L]] <- cbind(
      from[add_skip], sample(to2, sum(add_skip), replace = TRUE)
    )
  }
}

edge_mat <- do.call(rbind, edges_list)

node_names <- as.character(seq_len(id))
edge_df <- data.frame(
  from = as.character(edge_mat[, 1]),
  to   = as.character(edge_mat[, 2]),
  stringsAsFactors = FALSE
)
edge_df <- unique(edge_df)
edge_df <- edge_df[edge_df$from != edge_df$to, ]

node_df <- data.frame(
  name = node_names,
  cyclo = pmax(1, round(rgamma(id, shape = cc_shape, rate = cc_rate)))
)

tbl_graph(nodes = node_df, edges = edge_df, directed = TRUE)
}

```

3 Configurations

```

# DEFINE BENCHMARK CONFIGURATIONS #####

configs <- data.table(

```

```

config_id = 1:12,
n_entry   = c(2, 3, 3, 4, 5, 5, 6, 8, 10, 12, 15, 18),
n_mid_layer = c(10, 20, 40, 60, 80, 100, 130, 160, 200, 250, 300, 400),
n_layers   = c(3, 3, 3, 3, 4, 4, 4, 4, 5, 5, 6, 6),
n_sink     = c(4, 6, 8, 10, 12, 15, 18, 20, 25, 30, 35, 40),
p_extra    = c(0.05, 0.02, 0.008, 0.004, 0.002, 0.001, 0.0006, 0.0004,
               0.0003, 0.00025, 0.0002, 0.00015),
p_skip     = c(0.02, 0.01, 0.005, 0.003, 0.002, 0.001, 0.0005, 0.0003,
               0.0002, 0.00015, 0.0001, 0.00008)
)

configs[, expected_nodes := n_entry + n_mid_layer * n_layers + n_sink]
print(configs[, .(config_id, expected_nodes, n_layers, p_extra)])

```

```

##      config_id expected_nodes n_layers p_extra
##      <int>          <num>      <num>   <num>
##  1:         1             36         3 0.05000
##  2:         2             69         3 0.02000
##  3:         3            131         3 0.00800
##  4:         4            194         3 0.00400
##  5:         5            337         4 0.00200
##  6:         6            420         4 0.00100
##  7:         7            544         4 0.00060
##  8:         8            668         4 0.00040
##  9:         9           1035         5 0.00030
## 10:        10           1292         5 0.00025
## 11:        11           1850         6 0.00020
## 12:        12           2458         6 0.00015

```

4 Benchmark function

```

# BENCHMARK FUNCTION #####

run_benchmark <- function(cfg, N_ua = 2^8) {

  cat("\n=== Config", cfg$config_id,
      ": ~", cfg$expected_nodes, "expected nodes ===\n")

  # A) Generate graph -----

  t_graph <- system.time({
    g <- make_scalability_dag(
      n_entry   = cfg$n_entry,
      n_mid_layer = cfg$n_mid_layer,
      n_layers   = cfg$n_layers,
      n_sink     = cfg$n_sink,
      p_extra    = cfg$p_extra,

```

```

    p_skip      = cfg$p_skip,
    seed        = 123 + cfg$config_id
  )
})["elapsed"]

n_nodes <- igraph::vcount(g)
n_edges <- igraph::ecount(g)
cat("  Nodes:", n_nodes, " Edges:", n_edges,
    " (edges/node:", round(n_edges / n_nodes, 2), ")\n")

# B) all_paths_fun -----

t_paths <- system.time({
  out <- tryCatch(
    softwareRisk::all_paths_fun(
      graph      = g,
      alpha      = 0.6,
      beta       = 0.3,
      gamma      = 0.1,
      p          = 1,
      complexity_col = "cyclo"
    ),
    error = function(e) {
      cat("  all_paths_fun failed:", e$message, "\n")
      NULL
    }
  )
})["elapsed"]

if (is.null(out) || nrow(out$paths) == 0) {
  return(data.table(
    config_id = cfg$config_id,
    n_nodes = n_nodes, n_edges = n_edges,
    n_paths = NA_integer_, n_layers = cfg$n_layers,
    p_extra = cfg$p_extra,
    t_graph = t_graph, t_paths = t_paths,
    t_ua = NA_real_, t_total = NA_real_,
    top1_stability = NA_real_, top10_jaccard = NA_real_,
    mem_mb = NA_real_
  ))
}

n_paths <- nrow(out$paths)
cat("  Paths:", n_paths, "\n")

# C) uncertainty_fun: full parameter space (alpha, beta, gamma, p) -----

```

```

t_ua <- system.time({
  ua_out <- tryCatch(
    softwareRisk::uncertainty_fun(
      all_paths_out = out, N = N_ua,
      order = "first"
    ),
    error = function(e) {
      cat("  uncertainty_fun failed:", e$message, "\n")
      NULL
    }
  )
})["elapsed"]

# D) Rank stability from UA draws -----

compute_stability <- function(ua) {
  if (is.null(ua)) return(list(top1 = NA_real_, jaccard = NA_real_))

  paths_ua <- ua$paths
  n_p <- nrow(paths_ua)
  n_draws <- length(paths_ua$uncertainty_analysis[[1]])

  Pk_matrix <- do.call(rbind, paths_ua$uncertainty_analysis)
  rank_matrix <- apply(Pk_matrix, 2, function(col)
    frank(-col, ties.method = "average"))

  top1_per_draw <- apply(rank_matrix, 2, which.min)
  top1_stability <- max(table(top1_per_draw)) / n_draws

  k_top <- min(10, n_p)
  n_pairs <- min(500, choose(n_draws, 2))
  pi <- sample(n_draws, n_pairs, replace = TRUE)
  pj <- sample(n_draws, n_pairs, replace = TRUE)
  valid <- pi != pj
  pi <- pi[valid]; pj <- pj[valid]

  jac <- vapply(seq_along(pi), function(idx) {
    a <- order(rank_matrix[, pi[idx]])[1:k_top]
    b <- order(rank_matrix[, pj[idx]])[1:k_top]
    length(intersect(a, b)) / length(union(a, b))
  }, numeric(1))

  list(top1 = top1_stability, jaccard = mean(jac, na.rm = TRUE))
}

stab <- compute_stability(ua_out)

```

```

# E) Memory and timing -----

mem_mb <- as.numeric(object.size(out) + object.size(ua_out)) / 1e6
t_total <- t_graph + t_paths + t_ua

cat("  Time - graph:", round(t_graph, 2),
    "s | paths:", round(t_paths, 2),
    "s | UA:", round(t_ua, 2),
    "s | total:", round(t_total, 2), "s\n")
cat("  Top-1:", round(stab$top1, 3),
    " | Jaccard:", round(stab$jaccard, 3), "\n")

data.table(
  config_id      = cfg$config_id,
  n_nodes        = n_nodes,
  n_edges        = n_edges,
  n_paths        = n_paths,
  n_layers       = cfg$n_layers,
  p_extra        = cfg$p_extra,
  t_graph        = t_graph,
  t_paths        = t_paths,
  t_ua           = t_ua,
  t_total        = t_total,
  top1_stability = stab$top1,
  top10_jaccard  = stab$jaccard,
  mem_mb         = mem_mb
)
}

```

5 Run benchmark

```

# RUN (PARALLEL) #####

set.seed(123)

n_cores <- max(1L, parallel::detectCores() - 2L)
cat("Running", nrow(configs), "configs on", n_cores, "cores\n")

## Running 12 configs on 8 cores

results_list <- parallel::mclapply(seq_len(nrow(configs)), function(i) {
  run_benchmark(configs[i])
}, mc.cores = n_cores)

results_list <- Filter(Negate(is.null), results_list)
results <- rbindlist(results_list)

```

```
cat("\n\n=== RESULTS ===\n")
```

```
##
##
## === RESULTS ===
## 668 expected nodes ===
##   Nodes: 36  Edges: 38  (edges/node: 1.06 )
##   Nodes: 194 Edges: 228 (edges/node: 1.18 )
##   Nodes: 337 Edges: 362 (edges/node: 1.07 )
##   Nodes: 131 Edges: 149 (edges/node: 1.14 )
##   Nodes: 668 Edges: 671 (edges/node: 1 )
##   Nodes: 69  Edges: 87  (edges/node: 1.26 )
##   Nodes: 420 Edges: 439 (edges/node: 1.05 )
##   Nodes: 544 Edges: 562 (edges/node: 1.03 )
##   Paths: 24
##   Paths: 66
##   Paths: 108
##   Paths: 159
##   Paths: 207
##   Paths: 237
##   Time - graph: 0 s | paths: 0.05 s | UA: 0.28 s | total: 0.33 s
##   Top-1: 0.352  | Jaccard: 0.664
##
## === Config 9 : ~ 1035 expected nodes ===
##   Paths: 305
##   Paths: 358
##   Nodes: 1035 Edges: 1061 (edges/node: 1.03 )
##   Time - graph: 0 s | paths: 0.13 s | UA: 0.65 s | total: 0.78 s
##   Top-1: 0.387  | Jaccard: 0.345
##
## === Config 10 : ~ 1292 expected nodes ===
##   Nodes: 1292 Edges: 1334 (edges/node: 1.03 )
##   Time - graph: 0.01 s | paths: 0.19 s | UA: 1.06 s | total: 1.25 s
##   Top-1: 0.613  | Jaccard: 0.569
##
## === Config 11 : ~ 1850 expected nodes ===
##   Nodes: 1850 Edges: 1907 (edges/node: 1.03 )
##   Paths: 547
##   Time - graph: 0.01 s | paths: 0.27 s | UA: 1.56 s | total: 1.84 s
##   Top-1: 0.469  | Jaccard: 0.31
##
## === Config 12 : ~ 2458 expected nodes ===
##   Nodes: 2458 Edges: 2543 (edges/node: 1.03 )
##   Paths: 701
##   Time - graph: 0.01 s | paths: 0.35 s | UA: 2.14 s | total: 2.5 s
##   Top-1: 0.344  | Jaccard: 0.353
##   Time - graph: 0.01 s | paths: 0.41 s | UA: 2.53 s | total: 2.95 s
```



```
## Top-1: 0.297 | Jaccard: 0.374
## Paths: 964
## Time - graph: 0.01 s | paths: 0.54 s | UA: 3.25 s | total: 3.8 s
## Top-1: 0.551 | Jaccard: 0.297
## Time - graph: 0.01 s | paths: 0.62 s | UA: 3.55 s | total: 4.19 s
## Top-1: 0.332 | Jaccard: 0.652
## Paths: 1306
## Time - graph: 0.01 s | paths: 0.93 s | UA: 5.29 s | total: 6.23 s
## Top-1: 0.422 | Jaccard: 0.322
## Time - graph: 0.01 s | paths: 1.24 s | UA: 6.53 s | total: 7.78 s
## Top-1: 0.387 | Jaccard: 0.273
## Time - graph: 0.02 s | paths: 1.65 s | UA: 9.1 s | total: 10.77 s
## Top-1: 0.297 | Jaccard: 0.189
## Time - graph: 0.03 s | paths: 2.08 s | UA: 12.2 s | total: 14.31 s
## Top-1: 0.57 | Jaccard: 0.281
```

```
print(results)
```

```
write.xlsx(results, "scalability_benchmark.xlsx")
```

6 Plots

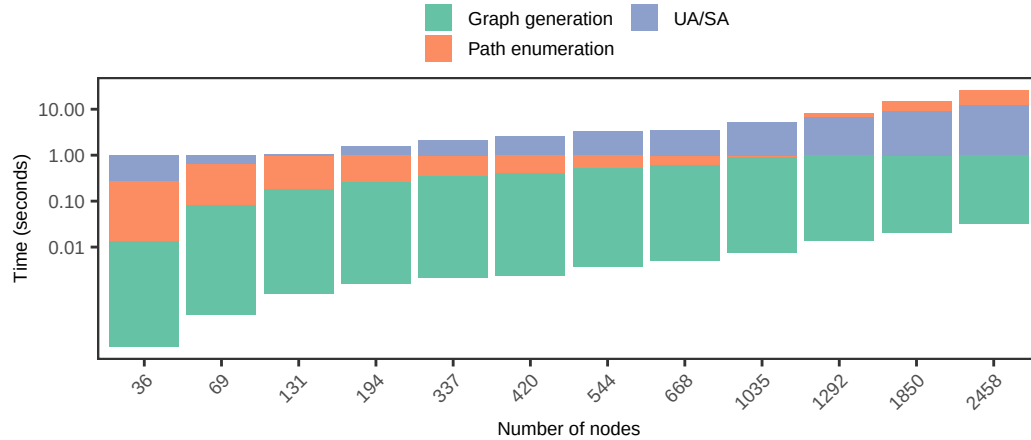
```
# A) Time by stage -----

time_long <- melt(
  results[!is.na(t_total)],
  id.vars = c("config_id", "n_nodes", "n_paths"),
  measure.vars = c("t_graph", "t_paths", "t_ua"),
  variable.name = "stage", value.name = "seconds"
)

time_long[, stage := fcase(
  stage == "t_graph", "Graph generation",
  stage == "t_paths", "Path enumeration",
  stage == "t_ua", "UA/SA"
)]

plot_time <- ggplot(time_long, aes(factor(n_nodes), seconds, fill = stage)) +
  geom_bar(stat = "identity") +
  scale_y_log10(breaks = c(0.01, 0.1, 1, 10, 100, 1000)) +
  labs(x = "Number of nodes", y = "Time (seconds)") +
  scale_fill_brewer(palette = "Set2", name = "",
    guide = guide_legend(nrow = 2)) +
  theme_AP() +
  theme(legend.position = "top",
    axis.text.x = element_text(angle = 45, hjust = 1))
```

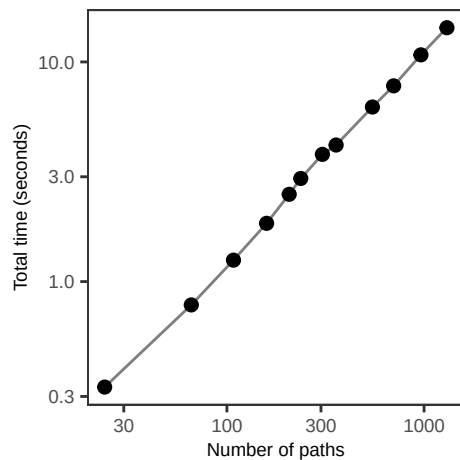
plot_time



B) Total time vs number of paths (log-log) -----

```
plot_time_paths <- results[!is.na(t_total)] %>%
  ggplot(., aes(n_paths, t_total)) +
  geom_point(size = 2) +
  geom_line(alpha = 0.5) +
  scale_x_log10() +
  scale_y_log10() +
  labs(x = "Number of paths", y = "Total time (seconds)") +
  theme_AP()
```

plot_time_paths



C) Stability -----

```
stability_long <- melt(
  results[!is.na(top1_stability)],
  id.vars = c("config_id", "n_nodes"),
  measure.vars = c("top1_stability", "top10_jaccard"),
  variable.name = "metric", value.name = "value"
```

```

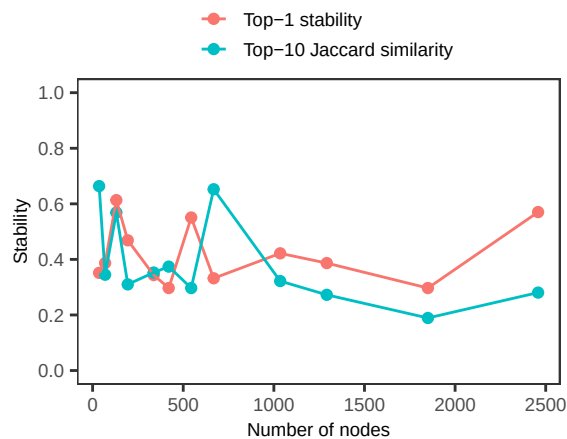
)

stability_long[, metric := fcase(
  metric == "top1_stability", "Top-1 stability",
  metric == "top10_jaccard", "Top-10 Jaccard similarity"
)]

plot_stability <- ggplot(stability_long,
                        aes(n_nodes, value, color = metric)) +
  geom_point() +
  geom_line() +
  scale_y_continuous(limits = c(0, 1), breaks = seq(0, 1, 0.2)) +
  labs(x = "Number of nodes", y = "Stability", color = "") +
  guides(color = guide_legend(nrow = 2)) +
  theme_AP() +
  theme(legend.position = "top")

plot_stability

```



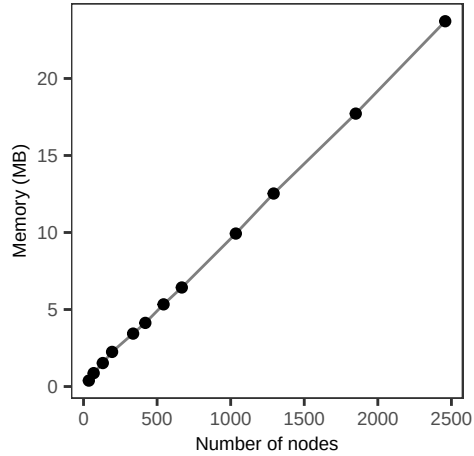
```

# D) Memory -----

plot_memory <- results[!is.na(mem_mb)] %>%
  ggplot(., aes(n_nodes, mem_mb)) +
  geom_point() +
  geom_line(alpha = 0.5) +
  labs(x = "Number of nodes", y = "Memory (MB)") +
  theme_AP()

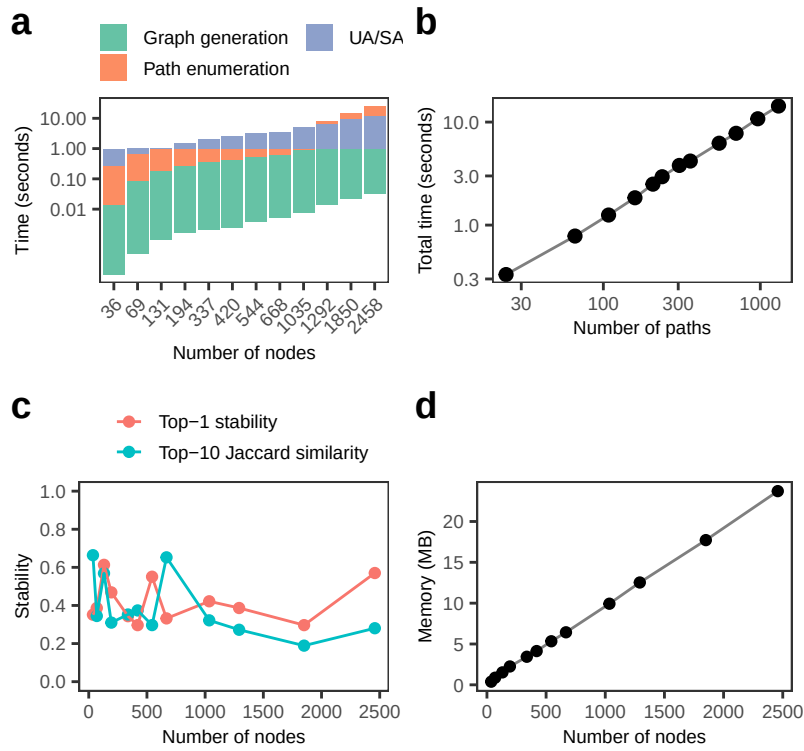
plot_memory

```



E) Merge -----

```
top_row <- plot_grid(plot_time, plot_time_paths, ncol = 2,
                     labels = c("a", "b"), align = "h", axis = "tb")
bottom_row <- plot_grid(plot_stability, plot_memory, ncol = 2,
                        labels = c("c", "d"), align = "h", axis = "tb")
plot_grid(top_row, bottom_row, ncol = 1)
```



7 Session information

SESSION INFORMATION

sessionInfo()

```
## R version 4.5.2 (2025-10-31)
## Platform: aarch64-apple-darwin20
## Running under: macOS Tahoe 26.3.1
##
## Matrix products: default
## BLAS:   /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework/Versions/A/Libraries/libBLAS.dylib
## LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: Europe/London
## tzcode source: internal
##
## attached base packages:
## [1] parallel stats      graphics  grDevices utils      datasets  methods
## [8] base
##
## other attached packages:
##  [1] benchmarkme_1.0.8   softwareRisk_0.2.0  sensobol_1.1.6
##  [4] igraph_2.2.1        tidygraph_1.3.1     cowplot_1.2.0.9000
##  [7] scales_1.4.0        openxlsx_4.2.8      lubridate_1.9.4
## [10] forcats_1.0.1       stringr_1.6.0       dplyr_1.1.4
## [13] purrr_1.2.1         readr_2.2.0         tidyr_1.3.2
## [16] tibble_3.3.1        ggplot2_4.0.1.9000  tidyverse_2.0.0
## [19] data.table_1.18.2.1
##
## loaded via a namespace (and not attached):
##  [1] benchmarkmeData_1.0.4 gtable_0.3.6        xfun_0.55
##  [4] ggrepel_0.9.6         lattice_0.22-7      tzdb_0.5.0
##  [7] vctr_0.7.1            tools_4.5.2         Rdpack_2.6.4
## [10] generics_0.1.4        pkgconfig_2.0.3     Matrix_1.7-4
## [13] RColorBrewer_1.1-3    S7_0.2.1            lifecycle_1.0.5
## [16] ineq_0.2-13           compiler_4.5.2      farver_2.1.2
## [19] tinytex_0.58          ggforce_0.5.0       codetools_0.2-20
## [22] graphlayouts_1.2.2    htmltools_0.5.9     yaml_2.3.12
## [25] pillar_1.11.1         MASS_7.3-65         cachem_1.1.0
## [28] iterators_1.0.14      viridis_0.6.5       foreach_1.5.2
## [31] tidyselect_1.2.1      zip_2.3.3           digest_0.6.39
## [34] stringi_1.8.7         labeling_0.4.3      polyclip_1.10-7
## [37] fastmap_1.2.0         grid_4.5.2          cli_3.6.5
## [40] magrittr_2.0.4        gggraph_2.2.2       withr_3.0.2
## [43] timechange_0.3.0      httr_1.4.8          rmarkdown_2.30
## [46] otl_0.2.0            gridExtra_2.3       hms_1.1.4
## [49] memoise_2.0.1         evaluate_1.0.5      knitr_1.51
```

```
## [52] rbibutils_2.4          doParallel_1.0.17    viridisLite_0.4.2
## [55] rlang_1.1.7            Rcpp_1.1.1           glue_1.8.0
## [58] tweenr_2.0.3           rstudioapi_0.17.1    R6_2.6.1
```

```
## Return the machine CPU -----
```

```
cat("Machine:    "); print(get_cpu())$model_name)
```

```
## Machine:
```

```
## [1] "Apple M1 Max"
```

```
## Return number of true cores -----
```

```
cat("Num cores:   "); print(parallel::detectCores(logical = FALSE))
```

```
## Num cores:
```

```
## [1] 10
```